KV Cache Management for LLM Inference: A Comparative Study under Realistic Serving Workloads

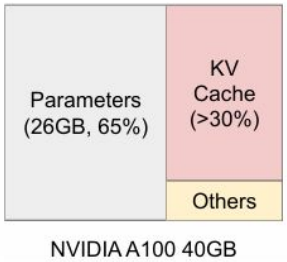
TEAM 31
Hung-Kai Huang · hh3164
Ting-Feng Huang · th3192
Sripad Karne · sk5695
Yipeng Wang· yw4623

GitHub: https://github.com/Willkczy/HPML-KV-Cache-Mangement
Tracking:

Motivation & Problem

Time budget: 1 minute.

Why this matters



KV cache scales linearly with context length and batch size, making it the dominant GPU memory bottleneck in LLM serving

At longer token contexts, the naive full-cache strategies exhaust GPU DRAM, forcing smaller batches and higher cost per token

Naive allocation causes severe memory fragmentation, limiting concurrent request throughput

Problem statement

Serving Qwen2.5-7B under long-context workloads on a single GPU: KV cache memory pressure degrades throughput and increases latency as context grows.

Goal: Quantify the memory–latency–quality trade-off across four KV cache strategies on identical hardware, targeting 40–60% memory reduction with controlled quality degradation.

Technical Approach — Model & Data

Time budget: ~1 min of the 3-min approach section.

Model

Qwen2.5-7B: decoder-only, 28 layers, GQA (4 KV heads / 28 Q heads), hidden size 3584, head dim 128, bfloat16, 128k context window

Full Cache & eviction methods via Hugging Face Transformers; Page Attention via vLLM

Dataset

Bucket	Dataset	Tokens	Generation	Quality
Short	MMLU	128-512	10 tok	Accuracy
Long QA	LongBench v2	8-16k	10 tok	Accuracy
Long explain	LongBench v2	8-16k	512 tok	Accuracy
Summarization	GovReport	4-16k	512 tok	ROUGE-L

Hardware

Insomnia cluster:

NVIDIA L40S

RTXA6000 (48 GB VRAM)

Technical Approach — Four KV cache management strategies

1. **Full Cache:** Baseline, keeps every key and value vector for every token in the context
2. **H2O:** Keep the most important tokens instead of the full cache.
3. **StreamingLLM:** Simple sink-plus-window strategy
4. **PageAttention:** vLLM’s memory management strategy

Technical Approach — Experiments Setups

Experiment 1 - controlled single-request benchmark

isolate the behavior of each strategy without interference from batching, queueing, or serving scheduler effects

- MMLU short: 5-shot MMLU, 128-512 tokens, generate 10 tokens
- LongBench short answer: 8k-16k tokens, generate answer as A/B/C/D
- LongBench explain: 8k-16k tokens, generate 512 tokens, let the model explain first then give answer as A/B/C/D -> measured with accuracy
- GovReport summarization: 4k-16k tokens, generate 512 tokens on summarization -> measured with ROUGE-L

Each method receives the same formatted prompt and returns the same output schema

Realistic Serving - realistic Concurrent Serving Benchmark

measure what happens when requests arrive over time and multiple requests are active simultaneously

- short: 50%, MMLU, 0-512 tokens, 8 output tokens
- medium: 30%, GovReport, 1k-4k tokens, 256 output tokens
- long: 15%, GovReport, 4k-16k tokens, 512 output tokens
- very_long: 5%, LongBench, 16k-32k tokens, 10 output tokens

Total: 500 requests, Poisson arrivals, seed=42. Arrival rate swept: 0.5 / 1.0 / 2.0 / 4.0 req/s. Using vLLM's AsyncLLMEngine

Measured Performance Metrics

Experiment 1

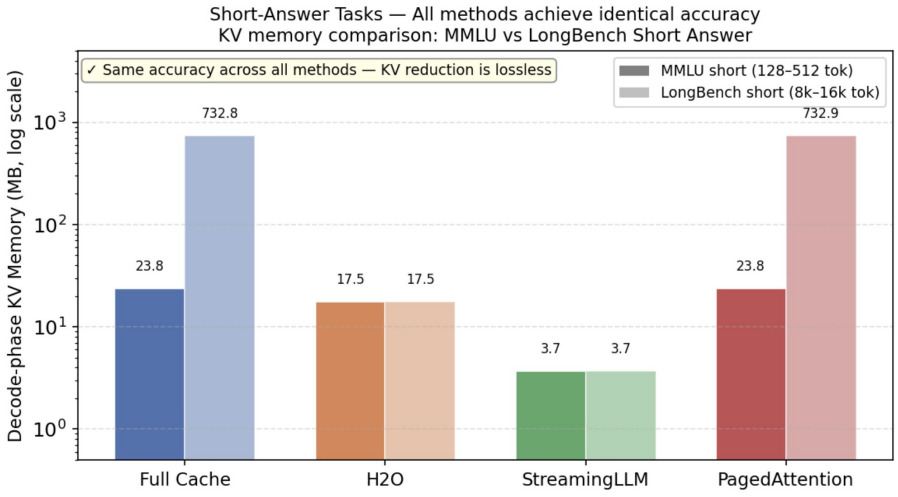
- latency
  - ttft_ms
  - decode_latency_ms: total_time_ms - ttft_ms
  - total_time_ms
- Throughput
  - throughput_tok_s: generated tokens / total time
- Memory
  - peak_kv_memory_mb
  - prefill_kv_memory_mb
  - n_oom
- quality
  - MMLU/LongBench -> accuracy
  - GovReport -> ROUGE-L

Concurrent Serving measures additional

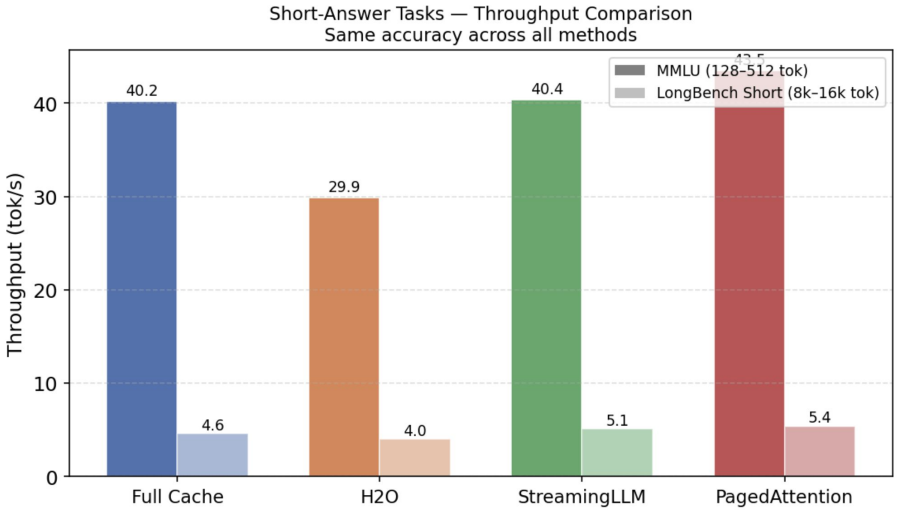
- request throughput req/s
- p50/p95/p99 TTFT & E2E latency
- service time
- peak/avg/p95 KV block utilization
- per-bucket quality

Results — Short-Answer Tasks

- *Quality unchanged ⟶ controlled setting confirmed*
- *Memory reduction confirmed (StreamingLLM 198×, H2O 42× on LongBench)*
- *System cost differences emerge (H2O 25% lower throughput)*



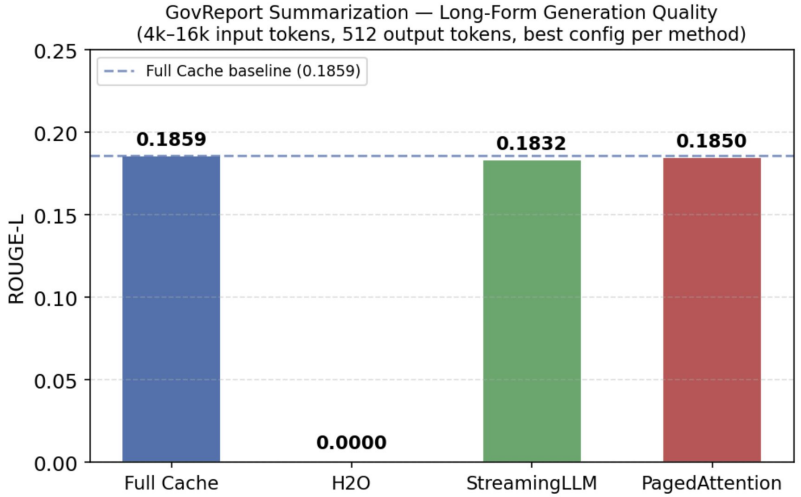
StreamingLLM: 6.4× less KV than Full Cache on MMLU | 198× less on LongBench | Zero accuracy loss



H2O is ~25% slower due to unfused eager attention kernel required for eviction scoring

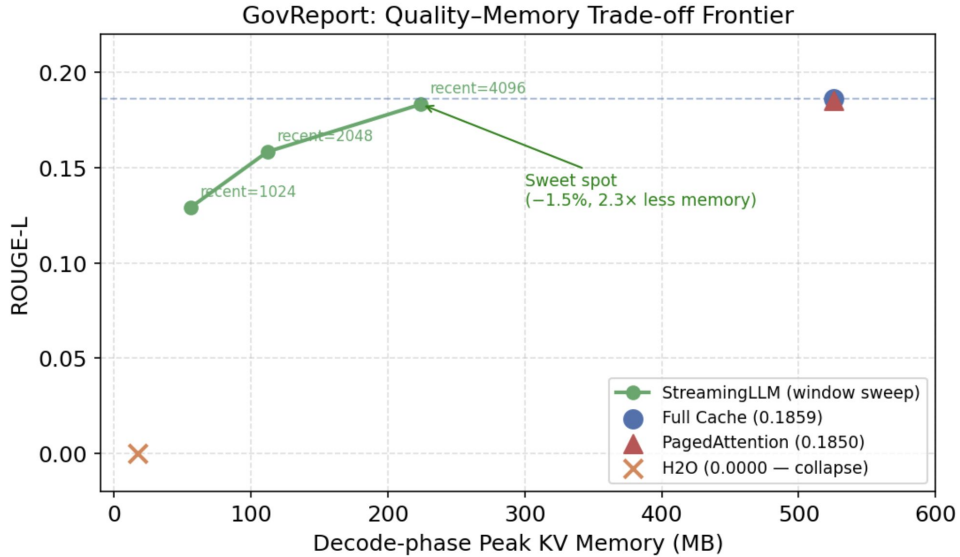
Results — Long-Form Generation

- *GovReport: 512 output tokens, 4k–16k input*
- *H2O: ROUGE-L = 0.000 at all budgets (128 to 2048 tokens)*
- *StreamingLLM (recent=4096): ROUGE-L = 0.1832 vs baseline 0.1859 (−1.5%)*
- *PagedAttention matches Full Cache*



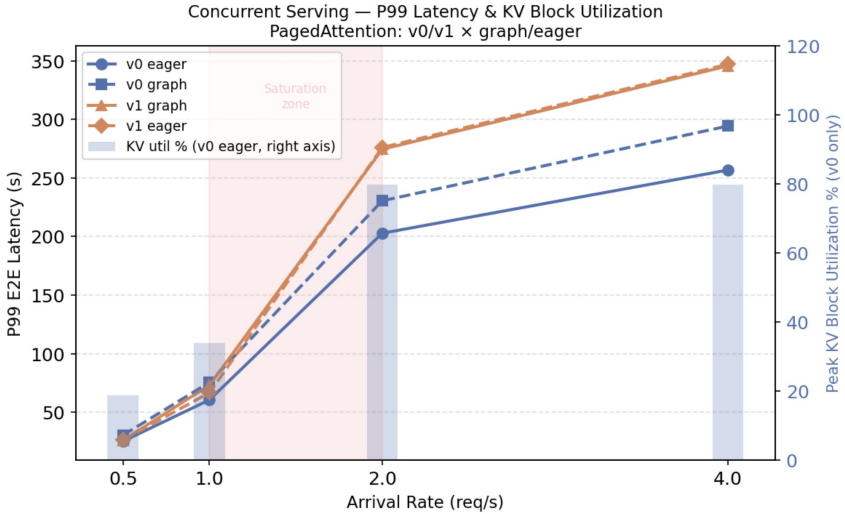
Results — Quality-Memory Trade-off

- *Window size is a tunable knob*
- *recent=1024 ⟶ −30% ROUGE-L, 9.3× less memory*
- *recent=2048 ⟶ −15% ROUGE-L, 4.7× less memory*
- *recent=4096 ⟶ −1.5% ROUGE-L, 2.3× less memory ⟵ sweet spot*



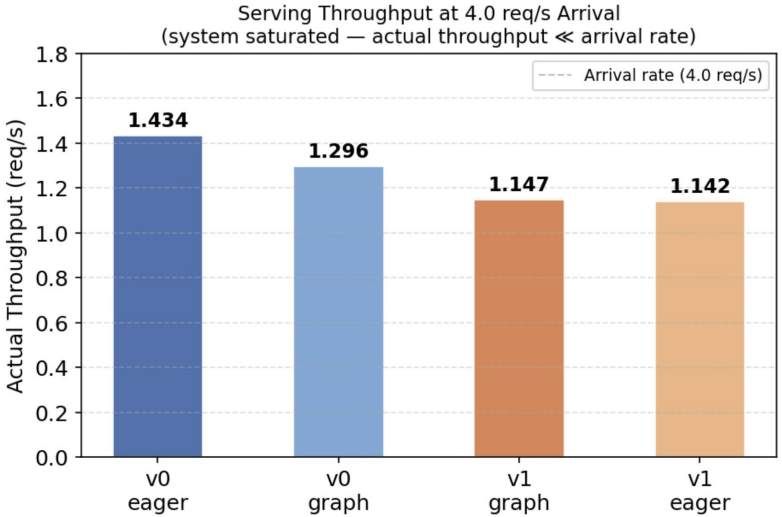
Results — Serving Results: Latency & KV Pressure

- *System saturates between 1.0 and 2.0 req/s*
- *v0_eager: best throughput (1.43 req/s), lowest P99 E2E*
- *KV block utilization peaks at 79.7% at saturation (v0 only)*
- *Low load (0.5 req/s): 18.7% KV util — headroom available*
- *High load (2.0+ req/s): 79.7% KV util — memory becomes bottleneck*
- *Quality identical across all configs*



Results — Quality-Memory Trade-off

- *Graph vs Eager: variable-length batches miss CUDA graph shapes → overhead without benefit*
- *v0 vs v1: chunked prefill in v1 (2048 tok/chunk) — 8 steps per long request vs 1*
- *Limitation: streaming_llm_vllm concurrent serving failed — CUDA race condition in block manager patches*
- *Takeaway: for mixed-length single-GPU serving, v0 eager + PagedAttention is the right config*



v0 eager is 24% faster than v1 — CUDA graphs hurt variable-length batches; v1 chunked prefill adds scheduling overhead

Lessons Learned

Time budget: 1 minute. What surprised you? What didn't work? Be honest.

What worked

- **StreamingLLM's sliding window scales gracefully.**

  Increasing the recent window from 1024 to 4096 tokens produced a smooth quality-memory Pareto curve, landing within 1.5% of full cache ROUGE-L at 2.3x less memory. Simple, predictable, easy to tune.

- **PagedAttention (v0 eager) is the best serving configuration.**

  Under a heterogeneous workload with extreme length variance (128 to 32k tokens), v0 eager hit 1.416 req/s at saturation, 10% higher than v0 with CUDA graphs. Disabling graphs avoided the shape-mismatch overhead that hurts mixed-length traffic.

- **Shared evaluation pipeline kept results comparable.**

  Every method returned the same `MethodOutput` schema, used the same prompts, and ran against the same trace (seed=42). This let us isolate KV cache behavior without worrying about differences in data loading or metric computation.

Next Steps

Time budget: 30 seconds. Be specific — vague "more experiments" is not next steps.

1. **Larger GPU to properly evaluate H2O**

   H2O needs raw attention scores to pick heavy hitters, which forces eager (unfused) attention since FlashAttention doesn't expose scores. Eager attention is O(n²) memory during prefill, so on a single 48GB A6000 we couldn't push budgets high enough on 4k-16k inputs to give H2O a fair shot, hence ROUGE-L = 0 on GovReport.

   An 80GB H100 or H200 would fit the eager prefill at meaningful budgets and actually test whether H2O's eviction policy holds up on long generation.

2. **LongBench explain accuracy was low across all methods.**

   Even the full cache baseline only hit 27.3% on explain-format prompts (vs 40.9% on short-answer). Makes it harder to draw clean conclusions about eviction quality on that benchmark.

3. **Multi-GPU tensor parallelism serving benchmark**

   All results are single A6000 (48GB), but v1 was built for multi-GPU. Running 2x/4x with TP would show if v1 pulls ahead with its intended parallelism, how KV pressure shifts across devices, and let us push past 34k context.

Teamwork & Contributions

Time budget: 30 seconds. Who did what. Required slide — graded individually.

Member	Primary contributions	Tools / artifacts
Ting-Feng (Fred)	Streaming LLM, Coordinating/Setting Up	StreamingLLM HF + vLLM implementations, window size sweep experiments
Hung-Kai (Will)	Full KV cache (HF), Coordinating/Setting Up	Full cache baseline, shared BaseMethod interface, data pipeline, evaluation metrics
Sripad	H20	H2O eviction method, attention score debugging and KV cache profiling
Kane	PagedAttention (vLLM)	PagedAttention vLLM integration, concurrent serving benchmark, Slurm infrastructure

Collaboration: regular [weekly / twice-weekly] meetings via [Zoom / in person], shared [Slack / Discord] channel, code review on every PR.

Thank you

Questions?

GitHub: https://github.com/Willkczy/HPML-KV-Cache-Mangement
Tracking:

Backup — Experimental Setup

Backup slide. Use for Q&A. Add more backup slides as needed.

Setting	Baseline	Optimized
Batch size	[32]	[64]
Precision	[FP32]	[BF16 + FP8]
Learning rate	[1e-4]	[1e-4 with warmup]
Sequence length	[2048]	[4096]
Epochs / steps	[3 epochs]	[3 epochs]
Hardware	[1× A100 80GB]	[1× A100 80GB]
Software stack	[PyTorch 2.5, CUDA 12.4]	[+FlashAttn-2, +bitsandbytes]

Profiler Evidence

Insert profiler trace screenshot, GPU timeline, or operator-level breakdown image.

[Insert profiler trace, Nsight timeline, or torch.profiler chrome trace screenshot here]

What to point at

- Bottleneck before optimization
- Kernel that disappeared / fused
- Idle gap that shrank
- Memory copy avoided

Demo / Visualization

Time budget: 1 minute. Live demo, animated comparison, or short recorded clip.

[Demo / GIF / Screen recording / Live notebook]

- *Have a fallback static image in case live demo fails*
- *Pre-load any models or data — do not wait for downloads on stage*
- *Narrate what to look at while it runs*